

UNIT V: Server Side Technology - PHP

Q1. What is the use of header() function in PHP?

The header() function in PHP is used to send a raw **HTTP header** from the server to the browser *before* any other output is sent.

Key Uses:

1. **Page Redirection:** This is the most common use. It tells the browser to load a different page.

```
<?php
// Redirect to another page
header("Location: new_page.php");
exit; // Always call exit() after a location header
?>
```

2. **Forcing File Download:** You can tell the browser to download content instead of displaying it.

```
<?php
header('Content-Type: application/octet-stream');
header('Content-Disposition: attachment; filename="myfile.zip"');
?>
```

3. **Setting Content Type:** Telling browser what kind of data is being sent (e.g., HTML, JSON).

```
<?php
// Tell the browser this is a JSON response
header('Content-Type: application/json');
?>
```

4. **Controlling Caching:** Telling the browser not to cache a page.

```
<?php
header('Cache-Control: no-cache, must-revalidate');
?>
```

Important Rule: The header() function **must** be called before any actual output is sent to the browser. This includes any HTML, echo statements, or even whitespace before the opening <?php tag.

Q2. What is the difference between require and include?

Both require and include are used to include and evaluate a specified PHP file within the current script. The main difference is how they handle errors if the file is not found.

- **include:** If the file specified in include is not found, PHP will generate a **Warning** (E_WARNING). The script will **continue to execute**, even after the warning.
 - **Use Case:** Best for optional files, like templates (headers, footers), where the rest of the page might still be able to function.
- **require:** If the file specified in require is not found, PHP will generate a **Fatal Error** (E_COMPILE_ERROR). The script will **stop** and will not execute any further.
 - **Use Case:** Best for essential files that the application *cannot* run without, such as database connections, core function libraries, or class definitions.

In summary: Use require for files that are critical. Use include for files that are not critical.

Q3. What is session? How we can register variables into session?

A session is a way to store information (in variables) for a single user, to be used across multiple pages of a website. Unlike cookies, session data is not stored on the user's browser. Instead, it is stored on the server.

When a session is started, PHP creates a unique session ID for that user and sends it to the browser to be stored in a cookie. When the user visits another page, the browser sends this ID back, and PHP retrieves the correct session data.

How to Register (Store) Variables into a Session:

There is a three-step process:

Step 1: Start the Session: You must call `session_start()` at the very top of every PHP page that needs to access session data, before any HTML or output.

```
<?php
session_start(); // Starts a new or resumes an existing session
?>
```

Step 2: Register (Store) Variables: Session variables are stored in a special superglobal array called `$_SESSION`. To "register" a variable, you simply add it as an element to this array.

```
<?php
session_start();
// Storing variables in the session
$_SESSION['username'] = 'tejashree';
$_SESSION['user_id'] = 123;
$_SESSION['is_logged_in'] = true;
echo "Session variables are set!";
?>
```

Step 3: Access Variables (on another page): On any other page (like `profile.php`), you again start the session and can then read the data from the `$_SESSION` array.

```
<?php
session_start(); // Resume the session
// Accessing the stored variables
if (isset($_SESSION['username'])) {
    echo "Welcome back, " . $_SESSION['username'];
} else {
    echo "Welcome, Guest!";
}
?>
```

Q4. Write short note on Cookie handling.

A **cookie** is a small piece of data that a server sends to a user's web browser. The browser stores this data (as a small text file) and sends it back to the server with every subsequent request. Cookies are used to remember information about the user.

Cookie Handling in PHP:

1. Creating (Setting) a Cookie: You use the `setcookie()` function. It must be called before any HTML or output is sent.

- **Syntax:** `setcookie(name, value, expire_time, path)`
- **Example:** This sets a cookie named "user" with the value "Tejashree" that will expire in 1 hour (3600 seconds).

```
<?php
$cookie_name = "user";
$cookie_value = "Tejashree";
$expiry_time = time() + 3600; // 1 hour from now
setcookie($cookie_name, $cookie_value, $expiry_time, "/");
?>
```

2. Reading (Accessing) a Cookie: You use the `$_COOKIE` superglobal array. It's good practice to first check if the cookie is set using `isset()`.

```
<?php
if (isset($_COOKIE['user'])) {
    echo "Welcome back, " . $_COOKIE['user'];
} else {
    echo "Welcome, Guest!";
}
?>
```

3. Deleting (Unsetting) a Cookie: To delete a cookie, you set it again using `setcookie()` but with an expiration date in the past.

```
<?php
// Set expiry time to 1 hour ago
setcookie("user", "", time() - 3600, "/");
?>
```

Q5. Discuss features of PHP.

PHP (Hypertext Preprocessor) is a popular server-side scripting language primarily used for web development. Its key features are:

1. **Server-Side Scripting:** PHP code is executed on the server, and the result (plain HTML) is sent to the browser. This makes it secure, as the user cannot see the source code.
2. **Open Source and Free:** PHP is completely free to use, download, and distribute. This lowers the cost of development.
3. **Cross-Platform:** It can run on all major operating systems, including Windows, Linux, and macOS.
4. **Wide Database Support:** PHP has excellent built-in support for a large number of databases, most notably **MySQL**, but also PostgreSQL, Oracle, SQL Server, and more.
5. **Easy to Learn (Simple Syntax):** PHP's syntax is similar to C and Java, making it easy for programmers to learn. It also has a low barrier to entry for beginners.
6. **Embeddable:** PHP code can be easily embedded directly into HTML files. You can mix HTML and PHP in the same file.

```
<h1>Welcome, <?php echo $username; ?></h1>
```

7. **Loosely Typed Language:** You do not need to declare the data type of a variable before using it. PHP automatically converts the type as needed.
8. **Large Community and Documentation:** PHP has been around for a long time, so it has a huge community, extensive documentation, and many frameworks (like Laravel and Symfony).

Q6. List out predefined classes in PHP.

PHP comes with many built-in, "predefined" classes that are part of its core and the Standard PHP Library (SPL). These classes are available in all scripts without needing to include any files.

Here are some common predefined classes:

1. **stdClass:** A generic "empty" class. It is often used to create a simple, empty object or for casting other types (like arrays) to an object.
2. **Exception:** The base class for all user-defined exceptions. It is used in try...catch blocks for error handling.
3. **DateTime:** The modern, object-oriented way to work with dates and times. It is much more powerful than the old date() and time() functions.
4. **PDO (PHP Data Objects):** A database abstraction layer. It provides a consistent, object-oriented interface for connecting to and interacting with various databases (like MySQL, PostgreSQL, etc.).
5. **mysqli:** A class (and set of functions) that provides an improved, object-oriented interface specifically for MySQL databases.
6. **DirectoryIterator:** Part of the SPL, this class provides a simple way to iterate (loop) over the contents of a filesystem directory.

Q7. How we get the browser properties using PHP?

PHP is a **server-side** language, which means it runs on the server, not in the browser. Therefore, it cannot *directly* access the user's browser properties like screen size, window dimensions, or installed plugins.

However, when a browser makes a request, it sends an **HTTP User-Agent string** as part of the request headers. PHP can read this string.

Method 1: Using the \$_SERVER Array: The \$_SERVER superglobal array contains all header information. The HTTP_USER_AGENT key holds the string sent by the browser.

```
<?php
// This gets the User-Agent string
$userAgent = $_SERVER['HTTP_USER_AGENT'];
echo $userAgent;
?>
```

- **Output (Example):** Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.4430.93 Safari/537.36

This string can be parsed to *guess* the browser (Chrome), operating system (Windows 10), etc., but it is not 100% reliable as it can be faked.

Method 2: Using get_browser()

PHP has a get_browser() function that can parse the User-Agent string and return a detailed object of the browser's properties.

- **Problem:** This function requires a server configuration file called browscap.ini. This file is often not set up on web servers, so the function will not work by default.

Conclusion: The only reliable information PHP can get is the \$_SERVER['HTTP_USER_AGENT'] string. For any other properties (like screen size), you must use a client-side language like JavaScript and then send that data to PHP (e.g., via an Ajax request).

Q8. Explain the mail() function in PHP to send an email

The mail() function is a built-in PHP function used to send a basic text email.

Function Syntax: bool mail(string \$to, string \$subject, string \$message, mixed \$additional_headers)

- **\$to:** The email address of the recipient(s).
- **\$subject:** The subject line of the email.
- **\$message:** The body (content) of the email.
- **\$additional_headers** (Optional but important): Used to add From, Cc, Bcc, or to send an HTML email.

Simple Example (Plain Text):

```
<?php
$to = "recipient@example.com";
$subject = "Test Mail";
$message = "This is a test email from my PHP script.";
// The 'From' header is essential, or the mail may be marked as spam
$headers = "From: webmaster@mysite.com";
if (mail($to, $subject, $message, $headers)) {
    echo "Email sent successfully!";
} else {
    echo "Email sending failed.";
}
?>
```

Example (HTML Email): To send an email formatted with HTML, you must set the Content-Type header.

```
<?php
$to = "recipient@example.com";
$subject = "My HTML Email";
$message = "
<html>
<body>
  <h1 style='color:blue;'>This is an HTML Email</h1>
  <p>This is a paragraph.</p>
</body>
</html>
";
// Set headers for HTML email
```

```
$headers = "From: webmaster@mysite.com\r\n";  
$headers .= "MIME-Version: 1.0\r\n";  
$headers .= "Content-Type: text/html; charset=ISO-8859-1\r\n";  
mail($to, $subject, $message, $headers);  
?>
```

Important Note: The mail() function relies on a mail server (like Sendmail or Postfix) being correctly installed and configured on the web server. On many shared hosting plans, this function is disabled or restricted to prevent spam. For reliable email sending, it is better to use a dedicated library (like PHPMailer) that connects to an SMTP server.

Q9. What are cookies? What can you do with it?

A cookie is a small text file that a web server sends to a user's browser. The browser stores this file on the user's computer. Every time the user visits that same website, the browser sends the cookie data back to the server.

This mechanism allows the website to "remember" information about the user.

What can you do with Cookies? (Uses):

1. **Session Management & Authentication:** This is the most common use. When you log in, a server can set a cookie with a unique ID. As you browse, the server reads this ID to know that you are still logged in. The "Remember Me" checkbox on login forms uses a long-lasting cookie.
 2. **Personalization (User Preferences):** Cookies can store user preferences, so the user doesn't have to set them on every visit.
 - *Examples:* Storing a site's language (e.g., lang=en), theme (e.g., theme=dark), or font size.
 3. **Tracking & Analytics:** Cookies are used to track user behavior.
 - *Examples:* Which pages you visit, how long you stay, and what you click on. This is used by services like Google Analytics.
 4. **Shopping Carts:** E-commerce sites use cookies to keep track of the items you have added to your shopping cart, even if you are not logged in.
-

Q10. Explain database connectivity in PHP with reference to MySQL.

To connect PHP to a MySQL database, you need to use a "database driver" or "extension." The old mysql_ functions are deprecated and unsafe. The two modern, recommended methods are:

1. **MySQLi** (MySQL Improved)
2. **PDO** (PHP Data Objects)

Here is a complete example of connecting, querying, and closing the connection using the **MySQLi** extension (in its object-oriented style).

There are 5 main steps:

Step 1: Create a Connection: You create a new mysqli object. You must provide the server (host), username, password, and database name.

Step 2: Check the Connection: Always check if the connection failed (e.g., wrong password, database server is down).

Step 3: Write and Execute the SQL Query: You write your SQL query as a string and use the query() method to run it.

Step 4: Fetch the Results: If the query was a SELECT statement, you get a "result set." You must loop through this set row by row to get the data. We use fetch_assoc() to get each row as an associative array (column name => value).

Step 5: Close the Connection: It is good practice to close the connection when you are finished.

Complete Example (using MySQLi):

```
<?php
// --- Database Credentials ---
$servername = "localhost";
$username = "root";
$password = "your_password";
$dbname = "my_database";
// --- Step 1: Create Connection ---
$conn = new mysqli($servername, $username, $password, $dbname);
// --- Step 2: Check Connection ---
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully!<br>";
// --- Step 3: Write and Execute SQL Query ---
$sql = "SELECT id, firstname, email FROM Users";
$result = $conn->query($sql);
// --- Step 4: Fetch the Results ---
if ($result->num_rows > 0) {
    // Output data of each row
    echo "<ul>";
    while ($row = $result->fetch_assoc()) {
        echo "<li>ID: " . $row["id"] . " - Name: " . $row["firstname"] . " - Email: " . $row["email"] . "</li>";
    }
    echo "</ul>";
} else {
    echo "0 results found";
}
// --- Step 5: Close Connection ---
$conn->close();
?>
```

Q11. How to create a database in MYSQL and how to create a table in it? Also explain How to insert data into it.

This is done using SQL (Structured Query Language) commands.

1. How to Create a Database: You use the CREATE DATABASE command. This creates a new, empty container for your tables.

```
CREATE DATABASE my_new_db;
```

After creating it, you must select it for use:

```
USE my_new_db;
```

2. How to Create a Table: You use the CREATE TABLE command. You must define the name of the table and the names and data types of each column.

- INT: Integer (whole number)
- VARCHAR(255): A variable-length string (text) up to 255 characters.
- PRIMARY KEY: This creates an index and ensures every row is unique.
- AUTO_INCREMENT: This makes the id column automatically generate a new, unique number (1, 2, 3...) for every new row.

Example (Creating a Students table):

```
CREATE TABLE Students (  
  id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  firstname VARCHAR(50) NOT NULL,  
  lastname VARCHAR(50) NOT NULL,  
  email VARCHAR(100),  
  registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. How to Insert Data into a Table: You use the INSERT INTO command. You specify the table, the columns you want to fill, and the values for those columns.

Example (Inserting one row):

```
INSERT INTO Students (firstname, lastname, email)  
VALUES ('Tejashree', 'Dumasia', 'tejashree@example.com');
```

Example (Inserting multiple rows):

```
INSERT INTO Students (firstname, lastname, email)  
VALUES  
('Amit', 'Sharma', 'amit@example.com'),  
('Priya', 'Singh', 'priya@example.com'),  
('Rahul', 'Gupta', 'rahul@example.com');
```

Q12. Explain of the following PHP/MySQL function: (i) mysql_query() (ii) mysql_error (iii) isset()

Important Note: The functions mysql_query() and mysql_error() are part of the original mysql_ extension, which is **deprecated** (old and no longer recommended) and has been **removed** from modern versions of PHP. The modern replacements are **MySQLi** and **PDO**.

(i) mysql_query(): This function was used to send an SQL query (like SELECT, INSERT, UPDATE) to the MySQL database.

- **How it worked:** `mysql_query($sql, $connection_link);`
- **Return Value:**
 - For SELECT queries, it returned a "result resource" (a reference to the data) on success, or FALSE on failure.
 - For INSERT, UPDATE, DELETE queries, it returned TRUE on success or FALSE on failure.

(ii) mysql_error(): This function was used to get the text of the error message from the *last* mysql_ function call that failed.

- **How it worked:** It was used for debugging. If `mysql_query()` returned FALSE, you would call `mysql_error()` to find out what went wrong (e.g., "Table 'Users' doesn't exist").
- **Example Usage:**

PHP

```
$result = mysql_query("SELECT * FROM NonExistentTable");
if (!$result) {
    // This would print the error message from MySQL
    die("SQL Error: " . mysql_error());
}
```

(iii) isset(): This is a core **PHP language function**, not a MySQL function.

- It checks whether a variable is **declared (set)** and is **not NULL**.
- It returns TRUE if the variable exists and is not NULL. It returns FALSE otherwise.
- **Why it's important:** It prevents "Undefined index" or "Undefined variable" errors. It is essential for checking if form data or session variables exist before using them.
- **Example Usage:**

PHP

```
// Check if a form was submitted
if (isset($_POST['username'])) {
    $username = $_POST['username'];
} else {
    $username = "Guest";
}
```

```
// Check if a session variable is set
if (isset($_SESSION['user_id'])) {
    // User is logged in
}
```

Q13. Can you explain the difference between MyISAM and InnoDB storage engines?

MyISAM and InnoDB are the two most common "storage engines" in MySQL. A storage engine is the underlying software that handles the creation, reading, updating, and deletion (CRUD) of data in a table.

InnoDB is the modern, default storage engine (since MySQL 5.5).

Feature	InnoDB	MyISAM
Transactions (ACID)	Yes (Supports COMMIT, ROLLBACK). This allows for safe, complex	No. All operations are final.

	operations.	
Locking	Row-Level Locking. Only the row being changed is locked. This is very good for performance when many users are writing data at the same time.	Table-Level Locking. The <i>entire table</i> is locked when a row is changed. This can be very slow if many users are writing.
Foreign Keys	Yes (Enforced). It supports relationships between tables.	No (Not enforced). It will accept the FOREIGN KEY syntax but will not enforce it.
Data Integrity	High. It is crash-resistant and can repair itself using its transaction logs.	Low. Data can be corrupted more easily in a server crash.
Use Case	Default for everything. Essential for applications requiring reliability, transactions, and high write performance (e.g., e-commerce, banking).	Read-Only Data. Sometimes used for read-heavy, low-write tables where speed is the only concern (e.g., logging, catalogs).

Q14. What does the JOIN statement do in MySQL? Explain the different types of joins.

A JOIN clause in SQL is used to combine rows from two or more tables based on a related column between them (a "foreign key"). This allows you to query data that is "normalized" or split across multiple tables.

For example, you might have a Users table and an Orders table. You can use a JOIN to get a list of all orders along with the name of the user who placed them.

Different Types of JOINS:

1. **(INNER) JOIN:** This is the most common type. It returns **only** the rows that have a matching value in **both** tables. *Analogy:* The intersection of two sets.
 - Example: `SELECT * FROM Users JOIN Orders ON Users.id = Orders.user_id;`
(This will only show users who have placed an order).
2. **LEFT JOIN (or LEFT OUTER JOIN):** It returns **all** rows from the **left** table (the first table mentioned). It also returns the matching rows from the **right** table. If there is no match in the right table, the columns from the right table will be filled with NULL.
 - Example: `SELECT * FROM Users LEFT JOIN Orders ON Users.id = Orders.user_id;`
(This will show all users, whether they have placed an order or not).
3. **RIGHT JOIN (or RIGHT OUTER JOIN):** This is the opposite of a LEFT JOIN. It returns **all** rows from the **right** table (the second table mentioned). It also returns the matching rows from the **left** table. If there is no match, the columns from the left table will be NULL.
 - Example: `SELECT * FROM Users RIGHT JOIN Orders ON Users.id = Orders.user_id;`
(This will show all orders, even if the user who placed it has been deleted).
4. **FULL OUTER JOIN** (Not in MySQL, but concept is important). It returns all rows when there is a match in **either** the left table or the right table. (In MySQL, you must simulate this by combining a LEFT JOIN and a RIGHT JOIN with the UNION operator).

Q15. What is the difference between `mysql_fetch_array` and `mysql_fetch_object`?

Important Note: Both of these functions are part of the **deprecated `mysql_` extension** and should not be used in modern code. The modern equivalents are methods on the MySQLi result object (e.g., `mysqli_result::fetch_array()` and `mysqli_result::fetch_object()`).

Both functions are used inside a loop to fetch a single row of data from a query result. The key difference is the **data type** of the returned row.

1. `mysql_fetch_array()`: This function returns the row as an **Array**. By default, it returns a single array that has **both** numeric indices (keys) and associative indices (column names).

```
// If the query was "SELECT name, city FROM users WHERE id = 1"
$row = mysql_fetch_array($result);
// You can access data by column name:
echo $row['name']; // e.g., "Tejashree"
// OR by numeric index:
echo $row[0]; // e.g., "Tejashree"
```

2. `mysql_fetch_object()`: This function returns the row as an **Object** (specifically, an instance of `stdClass`). The table's column names become the properties of the object. You must access the data using object notation (the arrow `->`).

```
// If the query was "SELECT name, city FROM users WHERE id = 1"
$row = mysql_fetch_object($result);
// You MUST access data by property name:
echo $row->name; // e.g., "Tejashree"
// This will NOT work:
// echo $row['name']; // Error
// echo $row[0]; // Error
```

In summary: `mysql_fetch_array()` returns an array, and `mysql_fetch_object()` returns an object.

Q16. Explain how MySQL database is managed.

Managing a MySQL database involves a wide range of tasks, from setup to optimization and maintenance. These tasks are typically performed using a few key methods:

- 1. Command-Line Interface (CLI):** This is the most direct and powerful way to manage MySQL.
 - The **mysql client** is a program that lets you log in to the server, run SQL queries, manage users, and run administrative commands.
 - *Example:* `mysql -u root -p`
 - Other utilities like `mysqldump` (for backups) and `mysqladmin` (for status/shutdown) are also used.
- 2. Graphical User Interfaces (GUIs):**
 - These are applications that provide a visual "point-and-click" interface for management, which is easier for many users.
 - **phpMyAdmin:** A free, web-based tool. It's very popular and included in many hosting packages (like XAMPP). It allows you to create tables, run queries, manage users, and export/import data, all from your browser.
 - **MySQL Workbench:** The official, free desktop application from Oracle (who owns MySQL). It provides advanced features for database design (modeling), development, and administration.

- **Other tools:** DBeaver, HeidiSQL, and Navicat are other popular desktop GUI clients.

3. SQL Commands:

- The database is managed *internally* using SQL.
- **DML (Data):** INSERT, UPDATE, DELETE
- **DDL (Structure):** CREATE, ALTER, DROP (for tables, databases)
- **DCL (Security):** GRANT, REVOKE (for managing user permissions)

4. Configuration File:

- The MySQL server's behavior (e.g., memory usage, port number, logging) is controlled by a configuration file named `my.cnf` (on Linux) or `my.ini` (on Windows). Administrators edit this file to tune the server's performance.

Q17. What is the use of phpMyAdmin in MySQL?

phpMyAdmin is a free, open-source, and very popular **web-based** graphical user interface (GUI) tool used for managing MySQL and MariaDB databases.

It is written in PHP and provides an easy-to-use interface that simplifies many common database tasks. Instead of writing complex SQL commands, you can often "point and click."

Key Uses and Features:

- **Database and Table Management:** You can visually create, rename, and drop (delete) databases and tables.
- **Data Manipulation:** You can browse, search, insert, edit, and delete rows of data directly in a grid.
- **SQL Query Execution:** It provides an SQL query window where you can write and run any SQL command and see the results.
- **Structure Management:** You can add, delete, or modify columns (fields) in a table, as well as manage indices and foreign keys.
- **User Management:** You can create new database users and manage their permissions (e.g., "this user can only SELECT from this table").
- **Import and Export:** This is a key feature. You can easily **export** (back up) a database to an .sql file and **import** (restore) it.
- **Server Monitoring:** It can show basic information about the server's status, running processes, and performance.

It is included in most web hosting control panels and local development packages (like XAMPP and WAMP), making it highly accessible.

Q18. Name MySQL operators

MySQL provides several types of operators to perform operations on data.

1. Arithmetic Operators:

- + (Addition)
- - (Subtraction)
- * (Multiplication)
- / (Division)
- % or MOD (Modulus - remainder)

2. Comparison Operators:

- = (Equal to)
- != or <> (Not equal to)
- > (Greater than)
- < (Less than)
- >= (Greater than or equal to)
- <= (Less than or equal to)

3. Logical Operators:

- AND or && (All conditions must be true)
- OR or || (At least one condition must be true)
- NOT or ! (Negates a condition)
- XOR (Logical exclusive OR)

4. Bitwise Operators:

- & (Bitwise AND)
- | (Bitwise OR)
- ^ (Bitwise XOR)

5. Special Operators (Pattern Matching and Set):

- **LIKE**: Used for pattern matching (e.g., WHERE name LIKE 'A%').
- **IN**: Checks if a value is within a set of values (e.g., WHERE city IN ('Pune', 'Mumbai')).
- **BETWEEN**: Checks if a value is within a range (e.g., WHERE age BETWEEN 18 AND 30).
- **IS NULL**: Checks if a value is NULL.
- **IS NOT NULL**: Checks if a value is not NULL.

Q19. How many different tables are present in MySQL?

This question likely refers to the different **types** of tables, which are defined by their **Storage Engine**. A user can create a virtually unlimited number of tables, but they must be of a type that MySQL supports.

A "storage engine" is the software component that MySQL uses to store, handle, and retrieve data for a table.

The most common types (storage engines) are:

1. InnoDB:

- This is the **default** storage engine in modern MySQL.
- It supports **transactions** (ACID compliant), **row-level locking**, and **foreign keys**. It is best for reliability and high-concurrency (many users).

2. MyISAM:

- This was the old default engine.
- It does *not* support transactions or foreign keys.
- It uses **table-level locking**, which can be slow for writing.
- It is very fast for read-only or read-heavy tables.

3. MEMORY:

- This engine stores the entire table in **RAM** (memory) instead of on disk.
- It is extremely fast but **volatile**: all data is lost when the server restarts.

- Used for temporary tables, lookup tables, or caching.
 - 4. **CSV:**
 - This engine stores its data in a plain text **Comma Separated Values (.csv)** file.
 - It is useful for importing or exporting data to be used with other applications (like Excel).
 - 5. **Archive:**
 - This engine is used for storing large amounts of un-indexed data in a highly compressed format.
 - It is used for "archiving" old data that is rarely accessed.
-

Q20. What are the advantages of using MySQL?

MySQL is one of the world's most popular open-source relational database management systems (RDBMS). Its key advantages are:

1. **Open Source and Free:** MySQL is free to use under the GNU General Public License (GPL). This significantly reduces the total cost of ownership as there are no expensive licensing fees.
 2. **High Performance:** MySQL is known for its speed, reliability, and scalability. It can handle very large databases and high-traffic websites.
 3. **Cross-Platform:** It is "platform-agnostic," meaning it can run on all major operating systems, including Linux, Windows, and macOS.
 4. **Easy to Use:** It is relatively simple to install, configure, and manage. Its popularity means there are many easy-to-use GUI tools (like phpMyAdmin and MySQL Workbench) available.
 5. **Large Community and Good Documentation:** As one of the most popular databases, it has a massive global community. This means it is easy to find tutorials, solutions to problems, and skilled developers.
 6. **High Security:** MySQL includes robust security features, such as user-based access control (privileges), password encryption, and support for secure (SSL) connections.
 7. **Scalability:** MySQL can be scaled up to handle massive amounts of data and high-volume operations, either by running on powerful servers or by using "replication" to spread the load across multiple servers.
 8. **Wide Integration:** It is the "M" in the **LAMP** (Linux, Apache, MySQL, PHP) stack and integrates perfectly with PHP, making it the default choice for web development for many years.
-